Reproducing LiTS Crosswords

A local-model reproduction and a terminal-state preservation fix

WU Yile | Student ID 1155269932 FTEC5660: Agentic AI for Business and FinTech

Experiment date: 22 October 2026 (Hong Kong).

Abstract

This study runs the authors' LiTS implementation on ten fixed mini crossword puzzles using a free local, quantized Qwen2.5-3B-Instruct model. An initial prompt experiment produces 60 actual episodes across three seeds, with 0% word accuracy in both conditions. Debugging reveals that a terminal error step makes the current-state accessor return the initial blank board, discarding earlier observations. A separate paired offline replay tests one isolated accessor fix on all 30 original-prompt episodes. It restores the observed board in 8 episodes and recovers 1.73% character accuracy, while word accuracy remains 0%. The work reproduces executable agent and environment behavior, identifies a reporting defect, and does not reproduce the paper's numerical performance.

1. Project and reproduction target

LiTS is project 3 on the assignment list. Its modular agent architecture separates action generation, transitions and rewards. This study reuses the original EnvChain, EnvGroundedPolicy and CrosswordsTransition implementations rather than rebuilding the agent. The task requires repeated observation, word selection and board updates, so it meets the assignment definition of a multi-step agentic system.

The numerical reference is the Crosswords chain row in Table 3 of Li and Tao (2026): 6.67% puzzle exact match and 10.33% word accuracy on 30 examples using Claude 3.5 Sonnet. This study uses ten fixed examples and a different model. Those reference numbers provide context only. Differences cannot be attributed solely to the model or treated as confirmation or rejection of the original result.

The original question was whether an explicit letter-consistency instruction improves final word accuracy. It did not. The debugging follow-up then tests an isolated state-accessor fix. This follow-up was designed after observing the initial failures and is disclosed separately. Its purpose is accurate state reporting, not an improvement in model reasoning.

2. Setup and experimental controls

Item	Recorded setting
Code	LiTS commit 1ef347bee7b3f2c10dc504342589b8c951794c2d
Data	First ten puzzles, IDs 0-9, from mini0505.json (156 puzzles in the source file)
Model	mlx-community/Qwen2.5-3B-Instruct-4bit; MLX, 4-bit weights
Model revision	4f83f8f146fdf28b512a06562b671d7af4fab457
Hardware	Apple Silicon arm64, 24 GiB system memory; existing user laptop
Runtime	Python 3.12.14; MLX 0.32.4; mlx-lm 0.29.1; Transformers 5.20.2
Decoding	Temperature 0.8, top_p 0.95, top_k 50, at most 32 generated tokens per call
Agent budget	10 steps; upstream three-attempt validation retry; depth-limit temperature forcing disabled in both conditions
Trials	Seeds 17, 42 and 2026; reset at the start of each paired episode
API and data access	No API keys, no paid API calls, no live web access during inference

The protocol was written before the measured runs. Puzzle 10 was reserved for the pilot and warm-up. The ten reported puzzle IDs were selected by position, with no filtering based on outcomes. The same seed initializes each paired baseline and modification episode. Condition order alternates deterministically across episodes. Divergent trajectories consume different random draws, so paired seeds are a variance-control convention rather than identical token samples.

Two package initializers use lazy imports to avoid pulling unused training and memory dependencies into the chain-only runtime. These infrastructure changes apply equally to both conditions. The submission includes an exact patch. The agent loop, validation, environment transitions, retry logic and official goal checker retain their pinned upstream implementations.

The lightweight environment uses Transformers 5.20.2 rather than the upstream full-install pin of 4.51.3. It supplies tokenization for the MLX adapter. This setup difference applies equally to the original and modified prompts. The exact resolved environment is included.

Model substitution: The underlying LLM differs from the paper, so results are not directly comparable. Local inference also changes hardware, quantization and latency conditions. Electricity and existing hardware costs are not priced.

3. Initial prompt experiment and evaluation

The modification appends the following instruction to the original crossword user prompt. No other experimental condition changes.

Letter-consistency rule: Every non-underscore letter in the chosen position is fixed by the current board. Your proposed word must match all of these letters in their exact positions. Choose another unfilled position if you cannot find a matching word.

The instruction uses only the visible board. Ground-truth answers remain in the evaluator. The original crossword template declares a goals variable but does not insert it into the prompt. The saved prompts allow this separation to be audited. The local adapter receives only formatted policy prompts, not the dataset answers.

Metric	Definition
Word accuracy (primary)	Number of correct horizontal and vertical final words divided by 10, averaged across ten puzzles per seed
Puzzle exact match	Fraction of puzzles with all 10 words correct
Character accuracy	Number of correct final board cells divided by 25
Invalid proposal rate	Rejected format or currently filled-position proposals divided by all model calls, including retries
Letter conflict rate	Valid-format proposals that disagree with at least one visible letter, divided by valid proposals
Efficiency	Model calls, generated tokens and inference seconds per puzzle episode
Terminal error rate	Fraction of episodes containing an error step that terminates the trajectory

For each metric, the analysis first computes a score over the ten puzzles for each seed, then reports the mean, sample standard deviation and sample variance across the three seed-level scores. The sample variance divides by n-1. Repeated runs on the same ten puzzles are not 30 independent puzzle samples. No statistical significance claim follows from three seeds. Final metrics on this page use the original accessor, including its observed terminal-error behavior.

The upstream validator checks action syntax, five-letter length and whether a position is unfilled. It does not enforce intersection consistency. Therefore the added instruction can change semantic behavior without changing executable action validity. Incorrect crossing words may overwrite earlier correct letters.

The independent scorer reconstructs all ten words directly from the final grid. It agrees with the upstream goal checker on every reported episode. Four hand-checked tests cover perfect, blank, malformed and one-letter-error grids. One wrong cell correctly breaks two words. Raw calls include invalid retries, so reported costs do not silently discard failures.

4. Initial experiment: measured negative result

Condition	Word accuracy	Puzzle exact match	Character accuracy
Paper chain reference (different model, 30 puzzles)	10.33%	6.67%	Not reported here
Local original prompt	0.00% +/- 0.00 pp	0.00% +/- 0.00 pp	0.00% +/- 0.00 pp
Local letter-consistency prompt	0.00% +/- 0.00 pp	0.00% +/- 0.00 pp	0.00% +/- 0.00 pp

Local entries show mean +/- sample standard deviation across three seeds. Percentage-point variability describes repeated runs on the fixed subset. The paper row is a contextual reference under different conditions.

Seed	Original word acc.	Modified word acc.	Paired difference
17	0.00%	0.00%	+0.00 pp
42	0.00%	0.00%	+0.00 pp
2026	0.00%	0.00%	+0.00 pp

Word-accuracy statistic	Original	Modified
Mean	0.00%	0.00%
Sample SD	0.000 pp	0.000 pp
Sample variance	0.000 pp^2	0.000 pp^2

The mean paired difference is +0.00 pp. All 60 episodes terminate after validation failures, so the prompt intervention does not establish an improvement. These zero final scores also motivated an audit of state handling. Full per-episode results and the six seed-level rows accompany the report as JSON and CSV. The follow-up below preserves these measurements rather than replacing them.

5. Inference effort and the terminal-state defect

Metric (per episode unless stated)	Original	Modified
Model calls	3.87 +/- 0.49	3.70 +/- 0.53
Generated tokens	25.10 +/- 5.83	29.90 +/- 5.67
Inference time, seconds	1.40 +/- 0.21	1.58 +/- 0.24
Invalid proposals	89.35% +/- 6.60 pp	89.75% +/- 6.70 pp
Letter conflicts among valid proposals	5.56% +/- 9.62 pp	0.00% +/- 0.00 pp
Terminal error episodes	100.00% +/- 0.00 pp	100.00% +/- 0.00 pp

Across all reported conditions, the model made 227 calls and generated 1,650 tokens. Measured inference time totaled 1.49 minutes. Paid API usage was USD 0. These timing values exclude downloading, model loading, warm-up and report preparation. Input length and retries can change runtime even though the nominal step limit stays fixed.

Illustrative original-prompt trajectory: puzzle 6, seed 17. Four valid placements precede a failed proposal. EnvChain appends an error step without next_state. The original accessor checks only that final step and returns the initial blank board. The preserved step history still contains the observed board. The example is selected by the largest character-accuracy difference, then lowest seed and puzzle ID.

Original accessor	Last observed board	Ground truth
_____	CARVD	AREFY
_____	APPLE	VALUE
_____	R____	ABOMA
_____	V____	LOPER
_____	DWELL	ETERN
Character acc. 0.00%	Character acc. 8.00%	100%

An underscore is an unfilled cell. This defect affects reporting after an error; it does not mean the model had solved the puzzle. In this example the original calls are unchanged and the recovered board still has zero correct complete words. The error flag must remain visible.

6. Isolated modification: preserve the last observation

The modification changes only EnvState.env_state. Instead of inspecting the final step alone, it searches the recorded steps in reverse order and returns the first nonempty next_state. It uses init_state only when no observation exists. The patch contains no new model, prompt, action policy or transition rule.

The follow-up protocol, PROTOCOL_STATE_FIX.md, was written after the initial experiment. All 30 original-prompt episodes enter a paired offline replay. Each pair has identical initial state, recorded actions, observations, terminal error and model-call count. The original accessor and a subclass implementing exactly the proposed fix read the same data. This is an offline software intervention, not 30 new live agent runs.

Replay outcome	Original accessor	Preserving accessor
Observed-placement episodes retaining their board	0 / 8	8 / 8
All episodes retaining their last observation	73.33%	100.00%
Mean final character accuracy	0.00%	1.73% +/- 0.83 pp
Mean final word accuracy	0.00%	0.00%
Terminal validation-error episodes	30 / 30	30 / 30
New model calls for replay	0	0

Seed	Original character acc.	Preserved character acc.
17	0.00%	2.40%
42	0.00%	2.00%
2026	0.00%	0.80%

Across three seed-level averages, preserved character accuracy has sample SD 0.833 pp and sample variance 0.693 pp^2. In the 22 episodes without an executed placement, both accessors correctly return the initial board. The recovered score reflects previously observed letters, not newly improved word proposals.

A separate deterministic test fills five rows with known benchmark answers, verifies the official checker reaches 100%, then appends a synthetic error step. The original accessor drops to the blank initial board, while the fix retains the solved board and the error flag. This validates environment behavior and the accessor mechanism. Because it uses the answer key, it is explicitly excluded from LLM performance results.

7. Debug diary and reproducibility audit

Blocker or risk	Resolution and evidence
Restricted network access	Downloaded the public repository and model with explicit network approval. Inference runs locally after assets are available.
Model download truncation	Checked expected safetensors byte length against the tensor header. Resumed via checked byte ranges and recorded a SHA256 manifest.
Large dependency graph	Stopped broad installation and used a chain-only environment plus two lazy-import adaptations. Both experimental conditions share these adaptations.
Metric implementation risk	Added an independent grid evaluator and four hand-checked tests. Checked every final score and per-call total against saved evidence.
Random outcomes	Prespecified three seeds, saved decoding settings and report seed-level variability.
Incomplete run recovery	Episode files support resumption. Interrupted call logs are preserved separately and excluded from completed episode totals.
Lost terminal observation	Traced the reset to EnvState.env_state after an error step. Paired replay and a deterministic answer-key software test isolate and verify the fix.

The repository contains the pre-experiment protocol, the pinned upstream revision, the runtime patch, install/run commands, the exact prompt change, model and dataset hashes, the resolved environment, completed episode logs and independently audited summaries. Each raw call records its prompt, response, decoding settings and token counts. No API keys are needed for this local setup.

8. Limits of the conclusions

The paper uses a stronger hosted model and 30 examples, while this study uses a quantized local 3B model and ten examples. The paper does not specify the exact Crosswords subset IDs or full chain setup in the experimental appendix. The experiment therefore cannot isolate the source of a paper-to-local accuracy gap. It also does not reproduce the paper's MCTS comparison or the separate action-diversity experiment.

The unsuccessful prompt study has an accuracy floor and a high invalid-proposal rate, so it cannot determine whether letter-consistency guidance helps a capable solver. The follow-up fix has stronger software evidence because identical observations isolate the accessor change. However, it repairs reporting only, and offline replay cannot establish a benefit to future live trajectories. Three seeds and ten related puzzles remain a small descriptive sample.

Local latency depends on the laptop workload and includes call overhead. The hardware and model differ from the paper, so its dollar costs and timing values should not be compared numerically with the local timing results.

9. Conclusions and future use

The authors' chain workflow executed 60 measured episodes with a free local model. Both prompt conditions obtained zero final word accuracy, so the paper's numerical performance did not reproduce in this setup. The logged trajectories reproduce executable action, transition and validation behavior and expose a terminal-state reporting defect. One isolated accessor modification restores the observed board in all 8 affected original-prompt episodes, recovering 1.73% mean character accuracy. Word accuracy and the validation-failure rate remain unchanged.

For future users, first validate the dataset and action format on an excluded puzzle. Pin the code and model, keep invalid attempts in cost totals, and inspect the last actual observation when an agent terminates with an error. Apply the included snapshot-preservation patch as a reporting correction. Improve the local model or output constraint separately, since combining that change with the accessor fix would obscure which intervention produced an effect.

Reproduction commands

python scripts/setup_runtime.py

python scripts/download_assets.py

python scripts/run_experiment.py --pilot

python scripts/run_experiment.py

python scripts/analyze_results.py

python scripts/replay_state_fix.py

python scripts/test_scoring.py

Run commands inside the documented environment from the repository root. The first setup step retrieves the exact upstream revision and applies the saved infrastructure patch. The README gives the installation commands and the supported Apple Silicon runtime. The evidence audit can run without loading a language model.

References

[1] Xinzhe Li and Yaguang Tao. 2026. LiTS: A Modular Framework for LLM Tree Search. ACL System Demonstrations, pp. 47-56. https://aclanthology.org/2026.acl-demo.5/

[2] LiTS authors. Source repository and included Crosswords benchmark. https://github.com/xinzhel/lits-llm. Exact revision recorded in config.json.

[3] Qwen Team. Qwen2.5-3B-Instruct. https://huggingface.co/Qwen/Qwen2.5-3B-Instruct. Local quantized distribution: https://huggingface.co/mlx-community/Qwen2.5-3B-Instruct-4bit. Exact revision and file hashes recorded in model_manifest.json.

Assistance disclosure

This submission package was prepared with AI assistance for code, experiment execution, analysis and writing. The included logs record actual local model calls. The student should review the code and conclusions, ensure the disclosure meets course policy, and personally deliver the recorded explanation.